Option / Maybe

Option · Parse positive port without null · Build with Waqar

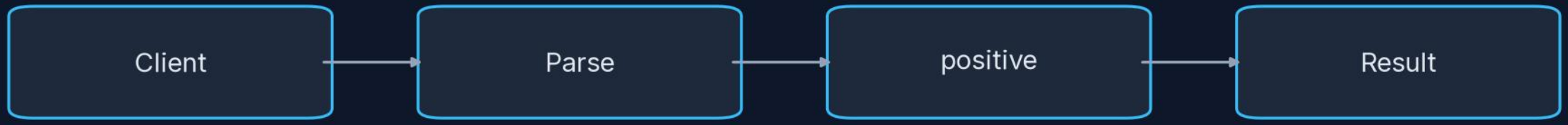
Today's Hook

- Problem: Parse positive port without null
- Pattern: Option — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Parse positive port without null
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Option: Parse positive port without null

Real-World Scenario

- Running example: Parse positive port without null
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.functional.option.problem;

import java.util.*;

public final class NaivePort {
  public static Integer parse(String s){ try { int p=Integer.parseInt(s); return p>0?p:null; } catch(Exception e){ return null; } }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.functional.option.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

final class Port {
  static OptionalInt parse(String s){
    try { int p=Integer.parseInt(s); return p>0? OptionalInt.of(p): OptionalInt.empty(); }
    catch(NumberFormatException e){ return OptionalInt.empty(); }
  }
  static String describe(OptionalInt p){ return p.isPresent()? "port="+p.getAsInt(): "missing"; }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.option.problem;
import java.util.*; import java.util.concurrent.*; import java.util.concurrent.atomic.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Option: parse positive port ===");
    System.out.println("--- naive ---"); System.out.println(NaivePort.parse("8080")); System.out.println(NaivePort.parse("-1"));
System.out.println("--- option ---");
System.out.println(Port.describe(Port.parse("8080")));
System.out.println(Port.describe(Port.parse("nope")));
System.out.println(Port.describe(Port.parse("0")));
  }
}
```

Walkthrough Output

```
=== Option: parse positive port ===
--- naive ---
8080
null
--- option ---
port=8080
missing
missing
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Option.

Related Patterns

- Contrast Option with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Option → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.